Examination Question Difficulty and Performance Analysis

Python Programming – Activity-Based Learning (ABL) Assignment

1. Problem Statement

Compare student answer strings against an answer key, calculate question-wise success rates, identify the most frequently missed questions, and classify question performance.

2. Objectives

• Compare each student's answer with the correct answer key. • Calculate correct and incorrect response counts. • Find question-wise success percentages. • Identify frequently missed questions. • Classify each question by performance.

3. Assumptions

The sample data contains five students and six multiple-choice questions. Answers are compared after trimming whitespace and converting to uppercase. A question is Easy when its success rate is at least 80%, Moderate when it is from 50% to below 80%, and Difficult when it is below 50%. These thresholds are an example policy and can be changed.

4. Algorithm

```
1. Store the correct answer for each question.
2. Store each student's submitted answers.
3. For every question, compare student answers with the key.
4. Count correct and incorrect responses.
5. Calculate success rate = correct responses / students * 100.
6. Classify the question using the stated thresholds.
7. Sort questions by incorrect response count.
8. Display question statistics and overall performance.
```

5. Flowchart (Text Representation)

```
START
  |
Read answer key and student answers
  |
For each question, compare all student answers
  |
Count correct and incorrect responses
  |
Calculate success percentage
  |
Classify as Easy / Moderate / Difficult
  |
Rank questions by number of incorrect answers
  |
Display question statistics and overall summary
  |
 END
```

6. Python Source Code

```
# Examination Question Difficulty and Performance Analysis
# Compare student answers with an answer key, calculate success rates,
# identify frequently missed questions, and classify question performance.

answer_key = {
    "Q1": "A",
    "Q2": "C",
    "Q3": "B",
    "Q4": "D",
    "Q5": "A",
    "Q6": "C",
}

students = {
    "S001": {"Q1": "A", "Q2": "C", "Q3": "B", "Q4": "D", "Q5": "B", "Q6": "C"},
    "S002": {"Q1": "A", "Q2": "B", "Q3": "B", "Q4": "C", "Q5": "A", "Q6": "D"},
    "S003": {"Q1": "C", "Q2": "C", "Q3": "D", "Q4": "D", "Q5": "A", "Q6": "C"},
    "S004": {"Q1": "A", "Q2": "C", "Q3": "A", "Q4": "B", "Q5": "D", "Q6": "C"},
    "S005": {"Q1": "B", "Q2": "D", "Q3": "B", "Q4": "D", "Q5": "A", "Q6": "A"},
}

def classify_question(success_rate):
    if success_rate >= 80:
        return "Easy"
    elif success_rate >= 50:
        return "Moderate"
    return "Difficult"
```

```
question_stats = {}
total_correct = 0
total_answers = len(students) * len(answer_key)

for qid, correct_answer in answer_key.items():
    correct_count = 0
    for answers in students.values():
        if answers.get(qid, "").strip().upper() == correct_answer:
            correct_count += 1
    success_rate = correct_count / len(students) * 100
    question_stats[qid] = {
        "correct": correct_count,
        "incorrect": len(students) - correct_count,
        "success_rate": success_rate,
        "classification": classify_question(success_rate),
    }
    total_correct += correct_count

print("EXAMINATION QUESTION DIFFICULTY & PERFORMANCE ANALYSIS")
print("=" * 82)
print(f"Students: {len(students)} | Questions: {len(answer_key)} | Total responses: {total_answers}")
print("Classification policy: Easy >= 80%, Moderate = 50%-79.9%, Difficult < 50%")
print("-" * 82)
print(f"{'Question':<12}{'Correct':>10}{'Incorrect':>12}{'Success %':>13}{'Classification':>19}")
print("-" * 82)

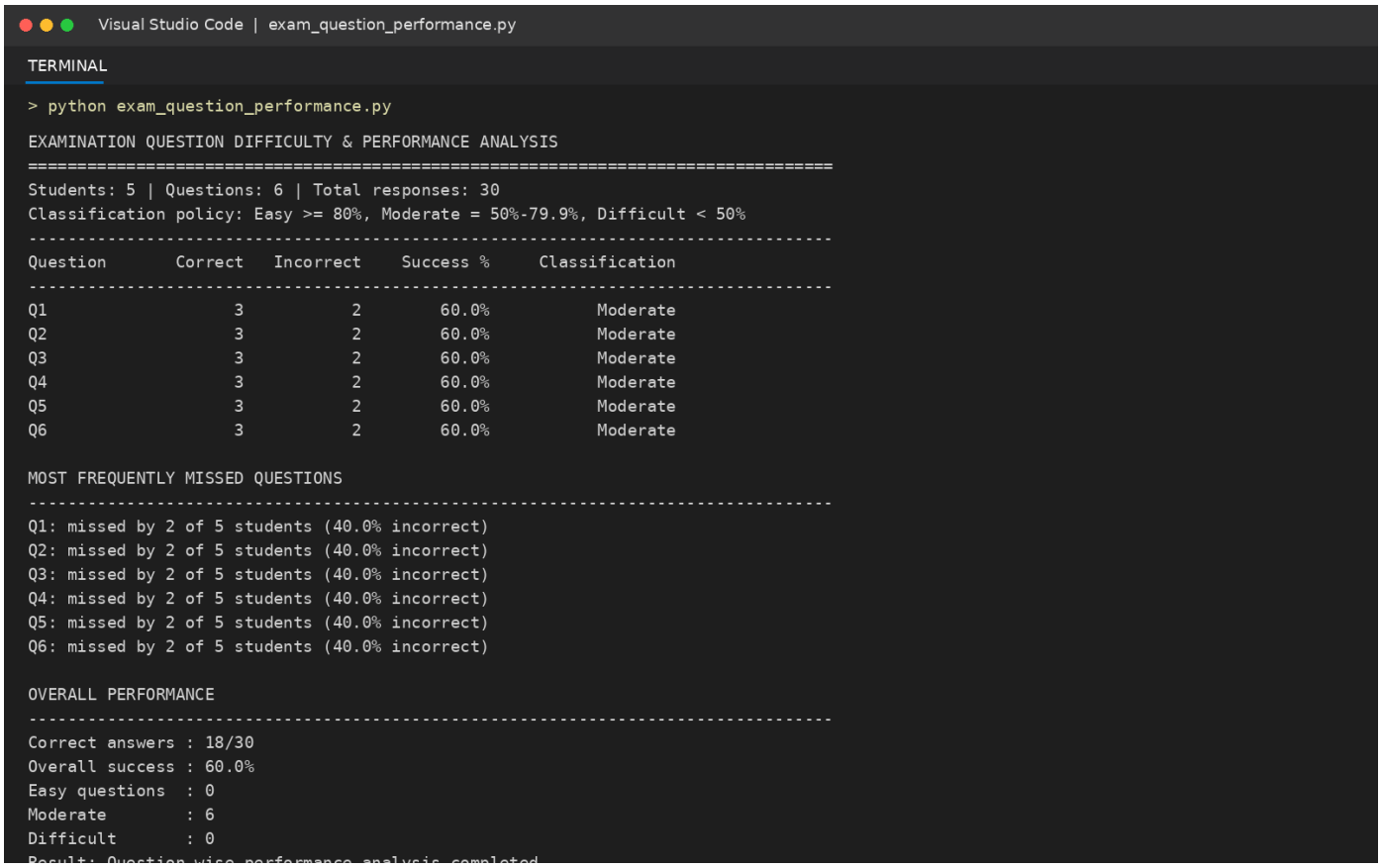
for qid, stats in question_stats.items():
    print(f"{qid:<12}{stats['correct']:>10}{stats['incorrect']:>12}"
          f"{stats['success_rate']:>12.1f}%{stats['classification']:>19}")

missed = sorted(question_stats.items(),
                key=lambda item: (-item[1]["incorrect"], item[0]))
print("\nMOST FREQUENTLY MISSED QUESTIONS")
print("-" * 82)
for qid, stats in missed:
    print(f"{qid}: missed by {stats['incorrect']} of {len(students)} students "
          f"({100 - stats['success_rate']:.1f}% incorrect)")

overall = total_correct / total_answers * 100
print("\nOVERALL PERFORMANCE")
print("-" * 82)
print(f"Correct answers : {total_correct}/{total_answers}")
print(f"Overall success : {overall:.1f}%")
print(f"Easy questions  : {sum(1 for s in question_stats.values() if s['classification'] == 'Easy')}")
print(f"Moderate        : {sum(1 for s in question_stats.values() if s['classification'] == 'Moderate')}")
print(f"Difficult       : {sum(1 for s in question_stats.values() if s['classification'] == 'Difficult')}")
print("Result: Question-wise performance analysis completed.")
```

7. Sample Execution Output Screenshot

This terminal-style screenshot shows the sample output. Run exam_question_performance.py in Visual Studio Code to reproduce it.



8. Test Cases

1. Exact match (for example S001, Q1 = A) -> correct.
2. Different answer (for example S002, Q2 = B while key is C) -> incorrect.
3. Lowercase/whitespace answer -> normalized before comparison.
4. Question success rate -> correct responses / 5 * 100.
5. Difficulty classification -> Easy >= 80%; Moderate >= 50%; Difficult < 50%.
6. Most-missed questions -> sorted by incorrect count, highest first.

9. Conclusion

The program compares student responses with the answer key, calculates question-wise success rates, highlights the most frequently missed questions, and classifies question performance using a clearly stated policy.